CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Dashboard Contents Calendar Assessments Syllabus

100%
Course progress
0% 100%

0 units left Completed

Resume >

January
February
March
April
May

← Prev Next →

Less More
2000 4000 6000

Description

Essentials of Data Science Laboratory - 2304102L

Upcoming tests

No upcoming exams in the next 7 days

1.1.1. Calculate Momentum

05:25

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
m is the mass of the object (in kilograms).
v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

calculate...

Submit

```
#Type Content here...
a=float(input(""))
b=float(input(""))
print(f"{a*b:.2f}kgm/s")
```

Terminal Test cases

1.1.1. Calculate Momentum

05:25

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
m is the mass of the object (in kilograms).
v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

calculate...

Submit

```
#Type Content here...
a=float(input(""))
b=float(input(""))
print(f"{a*b:.2f}kgm/s")
```

Terminal Test cases

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

1.1.2. Conditional Calculation Based on the Number of Digits

00:50

Write a Python program that accepts an integer n as input. Depending on the number of digits in n.

Constraints:

$1 \le n \le 999$

Input Format:

- The input consists of a single integer n.

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases +

Explorer

condition...

Submit

```python
#write your code here...
import math

# Accept an integer n as input
```

Average time 0.007 s 6.86 ms

Maximum time 0.012 s 12.00 ms

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 12 ms Debug

Expected output	Actual output
9	9
81	81

Test case 2 10 ms

Test case 3 5 ms

Terminal Test cases

Debugger

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

1.1.3. Days between Two Dates

05:47

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format $YYYY - MM - DD$.
- The second line contains the second date in the format $YYYY - MM - DD$.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases +

Explorer

noOfDay...

Submit

```
from datetime import date

#write your code here...
from datetime import datetime
```

Debugger

Average time	Maximum time
0.014 s	0.029 s
13.75 ms	29.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 29 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 12 ms

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

1.1.4. Reverse a Number

01:37

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases +

Explorer

reverseN...

Submit

Debugger

```python
#write your code here...
# Read input as an integer
num = int(input())
rev_num = 0

# Logic to reverse the number
while num > 0:
    remainder = num % 10
    rev_num = (rev_num * 10) + remainder
    num = num // 10

# Print the reversed number
print(rev_num)
```

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

1.1.5. Multiplication Table

00:21

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases +

Explorer

multiplica...

Submit

```
#write your code here...
# Read an integer from the user
number = int(input())
```

Average time	Maximum time
0.010 s	0.015 s
9.75 ms	15.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 11 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

Terminal Test cases

Debugger

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

2.1.1. List operations

00:23

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they

Explorer

listOps.py

Submit

```
# write the code..
numbers = []

while True:
```

Average time 0.087 s 87.33 ms

Maximum time 0.133 s 133.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 61 ms Debug

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 11	Integer: 11

Terminal Test cases

Debugger

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

2.1.2. Dictionary Operations

00:36

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.

Explorer

DictOper...

Submit

```
# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
```

Average time 0.022 s 22.25 ms

Maximum time 0.024 s 24.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 23 ms Debug

Expected output	Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}	Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11	11
Suresh	Suresh
After Insertion: {1: 'Amit', 2: 'Riya',	After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arju

Debugger

Terminal Test cases

< Prev Reset Submit Next >

3.1.1. Numpy Array Operations

00:35

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

numpyarr...

Submit

```
import numpy as np

# write your code here...
# Read rows and columns
```

Average time **0.015 s** 15.00 ms

Maximum time **0.025 s** 25.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 20 ms Debug

Expected output	Actual output
3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
9 10 11 12	9 10 11 12
[[1 2 3 4]	[[1 2 3 4]
[5 6 7 8]	[5 6 7 8]

Terminal Test cases

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

4.1.1. Pandas - series creation and manipulation

17:58

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases +

Explorer

seriesMa...

Submit

```python
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
s = pd.Series(numbers)
# Grouping by even and odd numbers and calculating the mean
grouped = s.groupby(s % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501040347@mitaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe

28:59

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.

Explorer

dataframe...

Submit

```
# import pandas as pd

# # Provided dictionary of lists
# data = {
```

Average time 0.119 s 119.00 ms

Maximum time 0.127 s 127.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 127 ms Debug

Expected output	Actual output
Original DataFrame:	Original DataFrame:
Name Age	Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
New name: Susan	New name: Susan

Terminal Test cases

Debugger

< Prev Reset Submit Next >

4.1.3. Student Information

31:46

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases +

studentin...

Submit

```python
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=
["Name", "Age", "Grade"])


# write your code here..

print("First five rows:")
print(data.head(5))
print("Average age:",round(data['Age'].mean(),2))
print("Students with a grade up to B")
print(data[data['Grade'].isin(['A','B'])])
```

Terminal Test cases

5.1.1. Stacked Plot

00:32

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

Sample Test Cases +

Explorer

stackedpl...

Submit

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
```

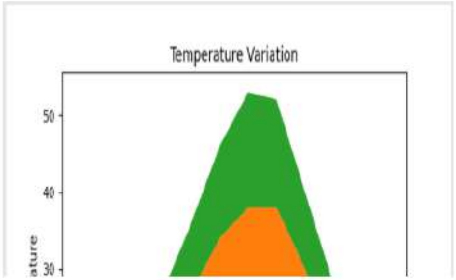
Average time 0.442 s 442.00 ms

Maximum time 0.442 s 442.00 ms

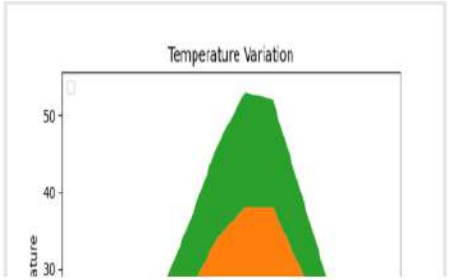
1 out of 1 shown test case(s) passed

Test case 1 442 ms Debug

Expected output



Actual output



Terminal Test cases

Debugger